

Deep Research: Migrating an Offline-First PWA from Centralized Server to Nostr Relays

Context

I'm migrating an offline-first PWA from a Flask-based centralized server to Nostr relays. The app manages encrypted chart data (Standard Celeration Charts — a specialized data visualization tool). The server currently handles sync, real-time collaboration, share links, account transfer, and subscription verification. The subscription layer is being eliminated entirely as part of this migration.

The app is genuinely offline-first. All chart creation, editing, rendering, and export happen locally in IndexedDB. The server is a convenience layer for multi-device sync and sharing. Solo users never need the server.

I will run my own relay for reliability, but **users must not be dependent on it**. The system must function across any set of compatible public relays.

Current Architecture (What We're Replacing)

Identity & Cryptography

- Identity derived from a **BIP39 12-word mnemonic** stored unencrypted in IndexedDB (`SCC_Identity` database).
- Private key is **never stored** — derived on every page load from the passphrase via PBKDF2 (salt `'ecdsa-signing'`, 100K iterations, SHA-256).
- **Curve: P-256 (secp256r1)** — NOT secp256k1. This is a critical incompatibility with Nostr.
- All crypto uses the **native Web Crypto API** exclusively. No external crypto libraries. Web Crypto does NOT support secp256k1.
- Custom BigInt P-256 point multiplication for PKCS8 key construction.

Encryption

- **Two-tier key scheme:**
 - **User master key (userKey):** derived from BIP39 passphrase via PBKDF2 (salt = SHA-256 of passphrase, 100K iterations). Held in memory only.
 - **Per-chart key (chartKey):** randomly generated AES-256-GCM key, stored as hex in IndexedDB per chart. NOT encrypted at rest.
- Encryption flow: plaintext chart → JSON.stringify → AES-256-GCM encrypt with chartKey → prepend 12-byte IV → hex-encode entire output.
- The userKey wraps/unwraps per-chart keys via AES-GCM key wrapping.
- **All crypto outputs are hex-encoded**, doubling payload size. This is a known inefficiency.

Blob Sizes

Metric	Value
Typical chart (100 data points)	50–150 KB plaintext → 100–300 KB hex-encoded ciphertext
Large chart (280 points, all features)	500 KB–1 MB plaintext → 1–2 MB hex-encoded
Theoretical max	~8 MB plaintext
Hex overhead	2×
Per-blob crypto overhead	28 bytes (12-byte IV + 16-byte auth tag)

Sync

- **Pull-on-load, not background polling.** Client sends local manifest (array of `{chart_uuid, updated_at}`) to `POST /api/sync`. Server returns only charts where `server.last_modified > client.updated_at`.
- **Strictly last-write-wins by timestamp.** No version vectors, no CRDTs, no merge logic. Server clamps timestamps to max 5 min in the future.
- Failed pushes queued in `localStorage` as JSON array of chart UUIDs. Drained on next page load or successful push. No backoff, no max retries.
- The `SYNC_SERVER_RECONNECTED` event is subscribed to but **never emitted** — dead code.

Real-Time Collaboration (WebSocket)

- Socket.IO 4.7.4 (client) / Flask-SocketIO 5.6.0 (server).
- Lightweight notifications only: `{chart_uuid, updated_at}`. No chart data over WebSocket.
- After notification, client fetches full chart via `GET /api/chart/{uuid}/shared`, decrypts, verifies, stores to IndexedDB.
- Reconnect: Socket.IO built-in exponential backoff (1s → 30s, infinite attempts). Visibility change listener triggers immediate reconnect.
- Server uses Socket.IO rooms (`chart:{uuid}`). Client joins room on connect, server broadcasts to room on chart update.

Sharing

- **Share link creation:** Generate 32-byte random `shareSecret` → derive `shareKey = PBKDF2(shareSecret, chartUuid)` → encrypt chart with `chartKey` → wrap `chartKey` twice (once with `userKey` for owner, once with `shareKey` for recipient) → POST all to server. Share URL = `https://host/chart/{chartUuid}/{shareSecret}` — `shareSecret` never sent to server.
- **Share link consumption:** Recipient opens URL → client extracts `shareSecret` → GET encrypted chart from server → derive `shareKey` from `shareSecret` → unwrap `chartKey` → decrypt chart → store locally with `shared: true` flag → strip `shareSecret` from URL via `replaceState`.
- **Edit links:** `acceptingEdits: true` flag inside encrypted data. Edit recipients can push updates. Signature verification is skipped for edit-link charts.
- **View-only links:** Recipient verifies ECDSA signature against owner's public key embedded in the encrypted data.
- **No explicit revocation.** `ShareLinks` have 14-day server-side TTL. Chart deletion cascades. "Unshare" creates a new chart with new UUID/key and abandons the old one.
- **Author updates propagate** via sync push → server stores → WebSocket notification → recipient fetches updated chart.

Account Transfer

- Sender generates UUID `linkId` + 32-byte `linkSecret` → derives AES key via `PBKDF2(linkSecret, linkId)` → encrypts `{passphrase, displayName}` → POSTs encrypted blob to server.
- Receiver scans QR code containing `https://host/sync/{linkId}/{linkSecret}` → GETs encrypted blob → derives key → decrypts → stores passphrase.
- **Server enforces 15-minute TTL.** Client shows countdown but it's cosmetic.
- Current code is **not one-time-use** despite design intent — doesn't delete after first fetch.

Server Authentication

- **None.** The server never verifies ECDSA signatures. All verification is client-side. `X-UserId` header is metadata only, trivially spoofable. Write operations gated by `@requires_subscription` decorator (being eliminated).
-

Target Architecture (What We're Building)

Event Model

NIP-33 parameterized replaceable events:

Chart sync (kind 30078):

json

```
{
  "kind": 30078,
  "pubkey": "<nostr pubkey derived from BIP39>",
  "created_at": "<unix timestamp>",
  "tags": [["d", "<chart-uuid>"]],
  "content": "<AES-256-GCM encrypted chart blob>",
  "sig": "<schnorr signature>"
}
```

Publishing a new event with the same **d** tag replaces the previous one on the relay.

Shared charts (kind 30079):

json

```
{
  "kind": 30079,
  "tags": [["d", "<chart-uuid>"], ["p", "<recipient-pubkey>"]],
  "content": "<encrypted blob + wrapped key>"
}
```

Recipient subscribes with filter **{kinds: [30079], #p: [own_pubkey]}**.

Real-time: Same REQ subscription stays open. Relays push matching events as they arrive.

Account transfer: NIP-44 encrypted DM to new device's pubkey.

Deletion: NIP-09 deletion event referencing **a** tag.

Relay Strategy

- I run my own relay as the primary, always-available relay.
- Users also connect to 2–4 public relays for redundancy.
- System must work if my relay goes down — public relays alone must suffice for sync.
- Publish to all connected relays. Read from all and merge/deduplicate.

What I Need Researched

1. secp256k1 in the Browser

The current system uses P-256 via Web Crypto API. Nostr requires secp256k1 (BIP-340 Schnorr signatures). Web Crypto does not support secp256k1.

- What is the current best library for secp256k1 in the browser? `@noble/secp256k1`, `@noble/curves`, or something else? What are the bundle sizes?
- What is the standard BIP39 → Nostr keypair derivation path? Is it BIP-44 `m/44'/1237'/0'/0/0`, or do most Nostr clients just use the raw seed/SHA-256 of the mnemonic? Is there a NIP that standardizes this?
- Can I derive a Nostr keypair deterministically from the same BIP39 mnemonic the app already uses, without changing the mnemonic itself? What is the exact derivation code?
- What are the performance implications of replacing Web Crypto ECDSA-P256 with a JS library for secp256k1 Schnorr signing? The app signs on every chart save.
- Since we're moving off Web Crypto for signing, should AES-256-GCM encryption also move to a library, or can it stay on Web Crypto while signing uses `@noble`? Any interop issues?

2. Nostr Client Libraries

- What is the current best Nostr client library for browser-based use? `nostr-tools`? Something else? What version is current and stable?
- Does it handle relay connection management (connect, reconnect, backoff) or do I need to build that?
- Does it handle NIP-33 replaceable events correctly (deduplication, replacement logic)?
- Does it support NIP-44 encryption out of the box?
- Does it support NIP-09 deletion?
- What is the bundle size? The app currently precaches all JS — this gets added to the service worker precache.

3. Blob Size vs Relay Limits

This is the highest-risk issue. Encrypted chart blobs are 100 KB–2 MB hex-encoded (50 KB–1 MB if we switch to base64).

- What are the actual event size limits on major public relays today? Not what NIPs say — what do relays actually enforce? Check: relay.damus.io, nos.lol, relay.nostr.band, relay.primal.net, purplepag.es, and any other high-reliability public relays.
- Is there a NIP or convention for large blob storage on Nostr? NIP-95 (file storage)? Blossom? Something else? What is the current adoption status?
- If I use NIP-95 or Blossom, do the major relays above support it?
- What is the practical approach people are using TODAY for storing 100KB–2MB encrypted blobs on Nostr? Not theoretical — what actually works in production?
- If chunking is needed: is there a NIP or convention for chunking large content across multiple events? How would reassembly work?
- Would switching from hex encoding to base64 encoding bring most charts under common relay limits? (Hex = $2\times$ plaintext, base64 = $1.33\times$ plaintext, so a 150 KB plaintext chart goes from 300 KB hex to 200 KB base64.)
- Is compression before encryption viable? What compression libraries work well in the browser for JSON data? What compression ratios are realistic for chart JSON?

4. Relay Selection & Management

- What is the current best practice for relay discovery and selection in browser-based Nostr clients?
- Is there a NIP for relay lists? (NIP-65?) How does it work? Should I publish a relay list event for each user?
- What are the most reliable public relays as of today for data persistence (not social media — I need relays that will persist replaceable events long-term)?
- What is the realistic data retention policy of public relays? Will they garbage-collect old replaceable events? After how long?
- If I run my own relay, what software should I use? strfry? nostream? Something else? What are the tradeoffs?
- How do I configure my own relay to accept larger events (up to 2 MB) while still being compatible with the Nostr protocol?
- What happens when a user publishes to 4 relays, 2 accept and 2 reject (e.g., size limit)? How should the client handle partial publish success?
- Is there a way to guarantee event persistence, or is "publish to N relays and hope" the best we can do?

5. NIP-33 Parameterized Replaceable Events

- Confirm: when I publish a kind 30078 event with `[d]` tag = chart UUID, and later publish another kind 30078 event with the same `[d]` tag and a newer `created_at`, do all compliant relays replace the old one? Or do some relays keep both?
- What happens if two devices publish conflicting updates with different `created_at` values? Does the relay keep the higher `created_at`? What if timestamps are equal?
- Is there a race condition risk if two devices publish near-simultaneously? How do relays handle this?
- Can I query a relay for "all kind 30078 events by pubkey X" efficiently? Is this a single REQ filter? What's the performance like with 50–200 charts?
- Can I query with a `since` filter to get only events newer than a timestamp (replicating the current sync manifest behavior)?

6. Real-Time Subscriptions

- When I have an open REQ subscription filtered on `{kinds: [30078], authors: [my_pubkey]}`, does the relay push new matching events in real-time as they arrive? Is this guaranteed behavior or relay-dependent?
- What is the latency profile? Socket.IO over my own server is ~50ms. What's typical for Nostr relay subscriptions?
- How does reconnection work? If the WebSocket to a relay drops and reconnects, do I need to re-send the REQ? Does `nostr-tools` handle this?
- Can I subscribe to multiple filters in one REQ (e.g., my own charts + charts shared with me)?
- Is there a keepalive/ping mechanism, or do I need to implement my own health check (currently `GET /api/health` every 10 seconds)?

7. Sharing Model

The current sharing model uses a URL fragment secret that never reaches the server. I need to replicate this on Nostr.

- **Option A:** Publish a kind 30079 event with a `(p)` tag for the recipient. The encrypted chart blob is in `(content)`, and the wrapped key (wrapped with a key derived from a secret in the URL fragment) is in a tag. The share URL = `(https://app/#/shared/<relay-hint>/<event-id>/<decryption-secret>)`. Is this sound?
- **Option B:** Use NIP-44 to encrypt the chart key directly to the recipient's pubkey, eliminating the URL secret entirely. This requires the recipient to have a Nostr identity. Is this simpler?
- **Option C:** Something else entirely?
- For edit-capable shares where the recipient can push updates: how does this work if the event is published under the original author's pubkey? The recipient can't publish a replaceable event with the same `(d)` tag under a different pubkey. What's the right model here? Separate events per collaborator that both sides merge?
- For anonymous share links (recipient doesn't have an account): NIP-44 won't work since there's no recipient pubkey. What's the best approach?
- How do I handle share link "expiry"? There's no server to enforce TTL. Is client-side expiry sufficient? Can I embed an expiry timestamp in the encrypted content?

8. Account Transfer Without a Server

Currently server-mediated: encrypted blob with 15-minute TTL.

- NIP-44 encrypted DM to the new device's pubkey: the new device would need a keypair before it has the passphrase. How is this bootstrapped? The new device doesn't have an identity yet — that's the whole point of the transfer.
- Alternative: publish an ephemeral encrypted event that the new device retrieves via a QR-code-scanned secret. But "ephemeral" on Nostr isn't guaranteed — relays may persist it. Is this acceptable given the content is AES-256-GCM encrypted with a secret not on the relay?
- Is there a NIP for ephemeral/temporary events that relays should not persist?
- What if I keep the current model but use a Nostr relay instead of my server? Publish an encrypted blob event, new device fetches it by event ID (from QR code), decrypt with the link secret. The blob is opaque. No TTL enforcement — but the content is encrypted and the secret is never on the relay. Is this good enough?
- Could I use a disposable Nostr keypair on the new device just for the transfer handshake?

9. Deletion

- NIP-09: how reliable is deletion across relays? Do major relays honor it?
- For NIP-33 replaceable events, do I reference the `a` tag (`30078:<pubkey>:<d-tag>`) or the event `id`?
- If a relay doesn't honor the deletion request, the content is still encrypted — is this an acceptable fallback?
- Is there a convention for "soft delete" where I publish a replacement event with empty/tombstone content?

10. Offline Resilience & Queue Management

- The current push queue is `localStorage`-based with no backoff. For Nostr, I need to publish to multiple relays. What's the recommended pattern for queuing failed publishes with per-relay tracking?
- If the user is offline and makes 10 edits to the same chart, I only need to publish the final state (since NIP-33 replaces). Should the queue deduplicate by `d` tag?
- What happens if the client publishes an event while offline (queued), then comes back online and another device has published a newer event in the meantime? The queued event has an older `created_at`. Will relays reject it because a newer replaceable event already exists? Or will they accept it and then the newer one "wins"?

11. Migration Path

- Can I run the Nostr system in parallel with the existing server during migration? Dual-write: publish to both server and relays, read from both, prefer newer.
- What's the simplest way to migrate existing charts from the server to relays? Bulk publish all charts as NIP-33 events?
- The curve change (P-256 → secp256k1) means existing ECDSA signatures become unverifiable under the new scheme. Since signatures are only verified client-side, can I just stop verifying old signatures and start signing with Schnorr going forward?
- The same BIP39 mnemonic will derive a DIFFERENT Nostr pubkey than the current P-256 pubkey. How do I handle identity continuity? The `X-User-Id` is currently SHA-256 of the P-256 public key.

12. Security Considerations

- Chart UUIDs in `[d]` tags are visible metadata. An observer can see how many charts a pubkey has, when they were last updated, and their UUIDs. Is this acceptable? Should I hash the UUIDs?
 - The encrypted blob size reveals approximate chart complexity. Is this a meaningful information leak?
 - Relays see which pubkeys are subscribing to which `[p]`-tagged events (shared charts). This reveals the social graph of collaboration. How significant is this?
 - With no server-side authentication, what prevents a malicious actor from publishing garbage events with the same kind and `[d]` tag under my pubkey? (Answer: they can't — Schnorr signature verification. But confirm this is enforced relay-side.)
 - NIP-44 encrypted DMs: are these truly private, or can relays see sender/recipient pubkeys?
-

Deliverables

For each section above, I need:

1. **The current state of the art** — what's possible today, not theoretical future NIPs.
2. **The simplest viable implementation** — fewest dependencies, least code, most proven path.
3. **Specific gotchas, limitations, or unsolved problems** I'll hit.
4. **Code examples or library references** where applicable.
5. **Relay-specific behavior differences** where they exist — don't assume all relays behave identically.